Python essentials week 3-4

Multidimensional lists: a list that contains other lists

```
#groceries = [
#    ["chips","choco","jelly"],
#    ["milk","apple juice","lemon juice"],
#    ["strawberry","mango","apple"]
#]
#print(groceries[1][1])
#print(groceries[0][0])
```

Bubble sort: is a simple sorting algorithm that repeatedly compares and swaps adjacent elements to arrange a list in order

Algorthim:

```
#mylist = [8,10,6,2,4]
#swapped = True

#while swapped:
 #   swapped = False
  #  for i in range(len(mylist) -1):
   #     if mylist[i] > mylist[i+1]:
    #        swapped = True
     #       mylist[i], mylist[i+1] = mylist[i+1], mylist[i]
```

```
#llist = [8, 4, 5, 6, 7, 9, 21, 3, 51, 6, 3, 4, 86]
#for i in range(len(llist) - 1):
   #for j in range(len(llist) - 1 - i):
        #if llist[j] > llist[j + 1]:
           #llist[j], llist[j + 1] = llist[j + 1], llist[j]

#print(llist)
```

#sort()

#reverse()

tuples is used to store multiple items in a single variable

tuple is an ordered and unchangeable collection

tuples are written with round brackets()

index starts from 0 by default

```
#my_tuple = (1,2,3,4,5,6,7,8,9)
#my_list= list(my_tuple)
#my_list[0]=99
#print(my_list)
```

we can just transform tuple to lists

dictionaries are used to store values in key:value

a dictionary is an ordered , changeable collection that does not allow duplicate keys

dictionaries are written with curly braces{}

a function is a set of commands collected in one place and executed when we call it

print() is also a function that is ready-made

why do we use functions?

Reduce repetition organize the code, easy maintenance and modification, share code between parts of the program

Functions either built in or user defined functions

User defined functions

Functions that are written by programmer to perform a specific task

We define them using the keyword def

```
#Def function_name(parameters)
	#Statement1
	#Statement2
#function_name(arguments)
```

Parameters can be empty

Or defined as values

Or can have default values (when no value is passed the default is used)

Positional arguments

Keyword arguments

Default arguments

Do not name a variable and a function with the same name

Do not call a function without it`s required argument or than its default

Do not call a function before creating it

Return stores the value

Exceptions:

Try: place the code that might cause an error

Except: handles the error if it occurs (it can have more than one except)

Else: executes only if there`s no error

Finally: executes no matter what, an error occurred or not

ValueError : invalid value

ZeroDivisonError : dividing by zero

TypeError :wrong data type in operation

KeyError : key not found in dictionary

IndexError : index not found in list/tuple